3rd Quarter Physics Orbit Simulation

Grade 11 Unified Project

Section A: Groups 3 & 8

March 20, 2026

Prepared by Group Members:

1. Amanuel Wondesen
2. Dawit Mulugeta Gebre
3. Hizkel Melese Dessie
4. Kaleb Million Bekele
5. Kidus Dawit Weldeyes
6. Mikiyas Mesfin Kebere
7. Muhamed Jemal Idris
8. Nawolin Tesema Borema
9. Negedeyesus Zewdu
10. Wako Gutu Gudina
11. Yonatan Alemu Bereka
12. Yoseph Kiflu Tafesse

1 Introduction

Analytical calculus provides an elegant framework for solving idealized physics problems by hand. However, real-world celestial mechanics often involves complex, nonlinear forces that cannot be easily resolved using standard algebraic kinematic formulas. The primary goal of this project is to model the complex dynamics of a satellite orbiting Earth using numerical integration.

By utilizing Python, specifically the `vpython` module, we developed a 3D visual simulation of orbital motion. Our investigation focuses on modeling three distinct celestial trajectories: circular orbits, elliptical orbits, and open escape trajectories. Rather than

relying on pre-calculated paths, our simulation continuously updates the physics frame-by-frame, demonstrating how fundamental gravitational principles give rise to the diverse orbits observed in astrophysics.

2 Physics Theory & Numerical Method

2.1 Newton's Law of Universal Gravitation

The driving force behind our simulation is Newton's Law of Universal Gravitation, which states that every point mass attracts every other point mass with a force acting along the line intersecting both points. The gravitational force $\vec{F}_g$ exerted on a satellite of mass m by a central body (Earth) of mass M is given by:

$$\vec{F}_g = -\frac{GMm}{|\vec{r}|^2}\hat{r} = -\frac{GMm}{|\vec{r}|^3}\vec{r} \tag{1}$$

Applying Newton's Second Law ($\vec{F} = m\vec{a}$), we find the instantaneous acceleration:

$$\vec{a} = -\frac{GM}{|\vec{r}|^3}\vec{r} \tag{2}$$

2.2 Numerical Integration: Euler's Method

To simulate motion, we must compute the changing position and velocity over time. Because the acceleration changes continuously as the position $\vec{r}$ changes, standard kinematic equations fail. Instead, we use Euler's method, advancing the simulation over infinitesimally small time steps, Δt:

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \Delta t \tag{3}$$

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_{n+1} \Delta t \tag{4}$$

2.3 The Importance of Time Step (Δt) and Stability

The accuracy of Euler's method heavily depends on the size of Δt. Because the method uses the instantaneous rate of change to approximate a curve, a large Δt causes the calculated trajectory to overshoot the true path. This artificially injects energy into the system, leading to numerical instability.

3 Simulation Results & Interpretation

Our Python script successfully rendered three distinct orbital classes by varying the initial parameters of the satellite.

1. **Circular Orbit:** By setting the initial velocity to exactly $v = \sqrt{GM/r}$ perpendicular to the position vector, the satellite maintained a constant altitude and speed.

2. **Elliptical Orbit:** When the initial velocity was set higher than the circular velocity but below the escape velocity, the orbit became eccentric, accelerating as it approached the perigee and decelerating at the apogee.

3. **Escape Trajectory:** By imparting an initial velocity equal to or greater than the escape velocity ($v = \sqrt{2GM/r}$), the satellite broke free from Earth's gravitational bound.

The physical meaning behind our simulation data reinforces the principle of Conservation of Mechanical Energy. The total energy ($E = K + U$) dictates the geometry of the orbit. By simply tweaking the initial velocity vector in our code, we observed profound changes in the system's macroscopic behavior.

4 Coding Challenges & Troubleshooting

Translating analytical physics into a functioning Python script presented our team with technical hurdles:

- **Energy Drift & Numerical Instability:** Our initial tests used a large Δt, causing the satellite to slowly spiral outward. We mitigated this by reducing Δt, which drastically improved the conservation of mechanical energy.

- **Division by Zero:** A common runtime crash occurred when calculating gravitational force if the magnitude of the distance r became zero. We implemented a conditional check to register a collision and safely halt the simulation.

- **3D Camera Alignment:** Managing the perspective in VPython was challenging. We solved this by dynamically updating the `scene.camera.follow` attribute.

5 Reflective Analysis

This project served as a bridge between theoretical and computational physics. In a textbook, equations yield perfect answers. In contrast, computational physics forces us to digitize continuous time into discrete steps, introducing truncation errors. Furthermore, we learned about the limitations of hardware, specifically floating-point arithmetic errors. This highlights why rigorous error checking is vital in real-world aerospace engineering.

6 Project Links & Resources

The complete Python source code for the VPython 3D visualization, alongside our presentation slides, can be accessed via the links below.

- **Public GitHub Code Repository:**
  https://github.com/ihsham3553-source/Physics

- **LaTeX Beamer Presentation Slides (Overleaf):**
  https://www.overleaf.com/read/jzzzqvrjfgjr#352b24

7 Visual Evidence

Below are screenshots generated directly from our computational model in GlowScript/VPython, demonstrating the different resulting trajectories based on our numerical integration loop.

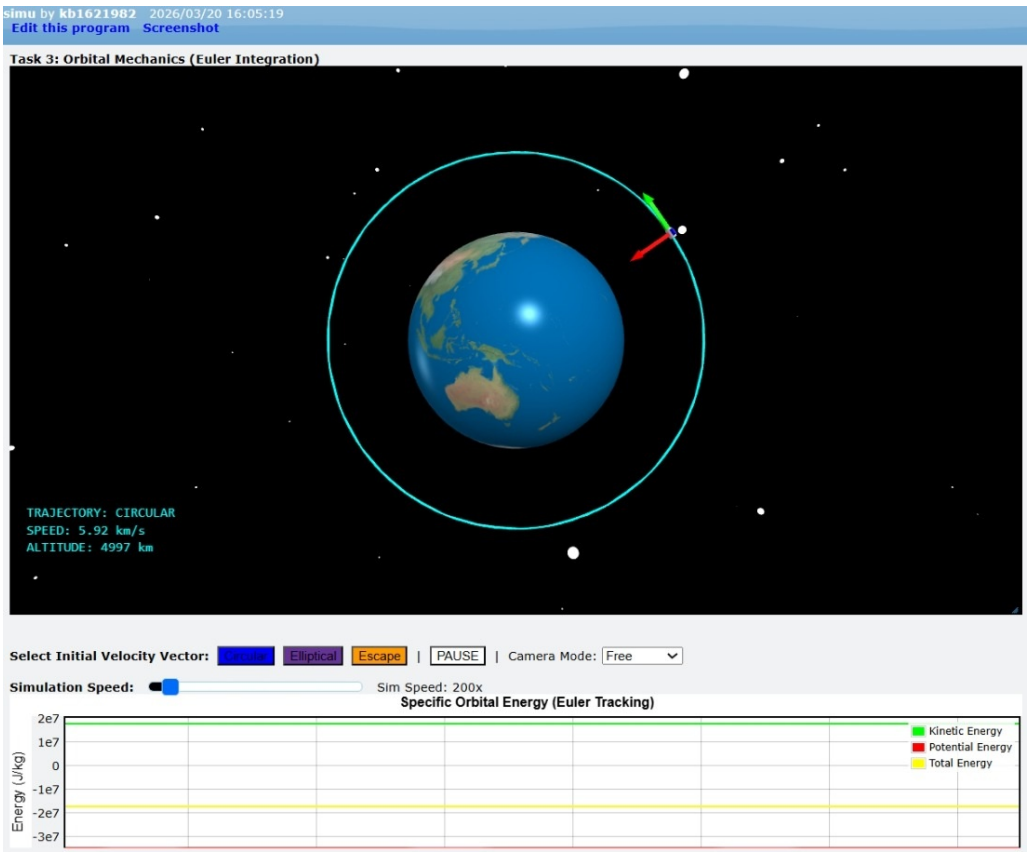

Figure 1: Simulation of a bound Circular Orbit, maintaining uniform altitude.

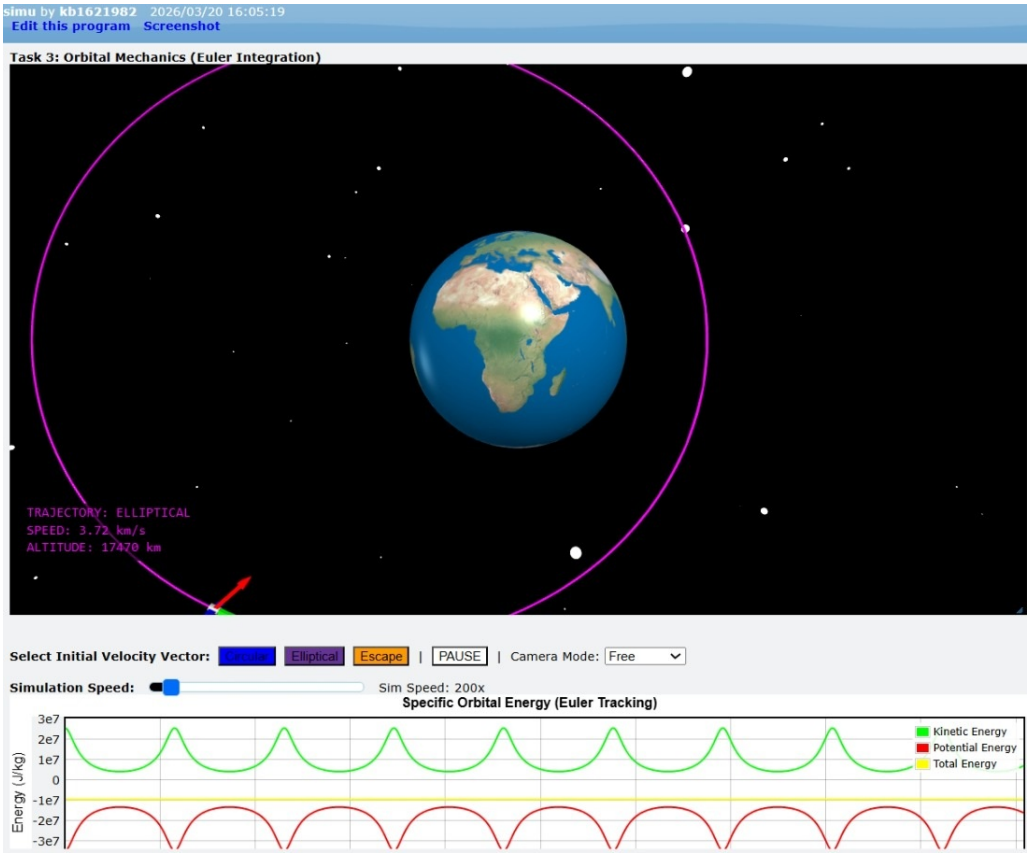

Figure 2: Simulation of a bound Elliptical Orbit, demonstrating varying velocity and altitude.

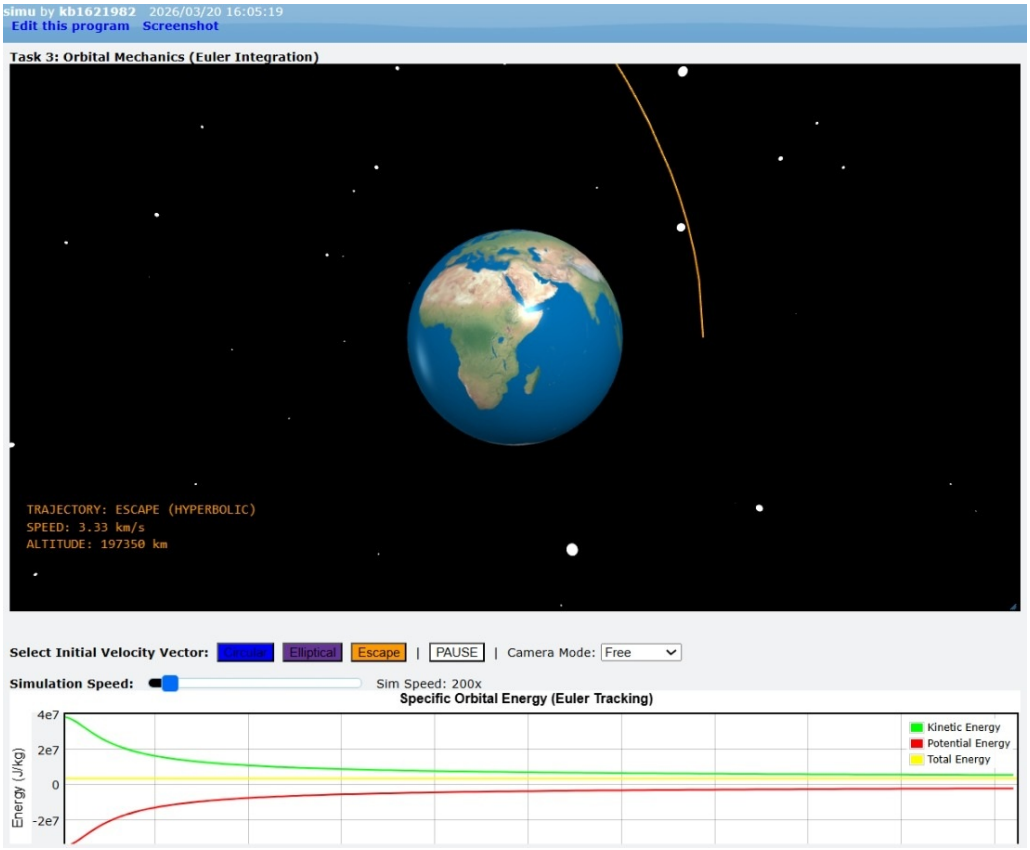

Figure 3: Simulation of an open Escape Trajectory, where kinetic energy overcomes the gravitational potential.